

BALLOO PROJECT BUILD QUESTIONNAIRE (PHASE 1-2)

Версия: 2.0.0

Дата: 2026-06-14

Цель: Собрать ответы для единой команды «собери и разработай проект»

НАЗНАЧЕНИЕ ЭТОГО ОПРОСНИКА

После заполнения вы сможете дать одну команду:

«собери и разработай проект»

И я выполняю:

- ☒ **Дострою документацию и codegen-AI context** — на основе ваших ответов
 - ☒ **Соберу единый файл документации и ТЗ** — все нюансы и требования в одном месте
 - ☒ **Напишу всю систему (Phase 1-2)** — 100% работоспособная, без глюков, без платных функций
 - ☒ **Расскажу как настроить и запустить на Ubuntu** — пошаговая инструкция для сервера
-

ЧТО УЖЕ ГОТОВО (НЕ ТРЕБУЕТ ОТВЕТОВ)

На основе существующей документации и кода я уже зафиксировал:

☒ Access Policy (39 файлов)

- 7 ролей (creator-superadmin, delegated-node-admin, company-staff, alpha-staff, alpha-volunteer, sandbox-operator, public-user)
- 5 групп узлов (A: privileged, B: company, C: alpha, D: sandbox, E: production)
- 20 узлов классифицированы
- creator-superadmin: Оберюхтин Иван Анатольевич (o8eryuhtin@yandex.ru)

☒ Auth Providers (20+ файлов)

- Phase 1: yandex-id, email-password, phone-3char-code
- Phase 2: gosuslugi, max, qr-code
- Cross-entry policy: автоматический вход на том же устройстве/браузере
- creator-superadmin: отдельная privileged identity

☒ Design System (16 файлов)

- 6 design invariants (border-radius: 0, border: 2px solid, 3 themes, etc.)
-

- 10 компонентов (Button, Input, Card, Modal, StatusBadge, Header, Footer, MetricCard, LogViewer, NodeStatusBlock)
- 8 screen patterns (auth, dashboard, settings, chat, nodeDetail, logs, health, error)

☒ Node Tree (20 узлов)

- Group A (4): projectgeneralsettings, kodegen, pilot-future, nodes-switcher
- Group B (2): workdocs, admin
- Group C (3): alpha, apps.alpha, 2commands
- Group D (9): working, api, files, docs, future, admin, workers, about, apps
- Group E (2): balloo.su, messenger

СЕКЦИЯ 1: СЕРВЕРНАЯ ИНФРАСТРУКТУРА (ОБЯЗАТЕЛЬНО)

Без этих ответов невозможна deployment инструкция для Ubuntu.

1.1 Серверные параметры

Вопрос	Ваш ответ
1.1.1 Какой OS сервер? (Ubuntu 20.04/22.04/24.04)	
1.1.2 Сколько серверов? (1 для начала или больше)	
1.1.3 Есть ли доменное имя? (какое)	
1.1.4 Есть ли SSL сертификат? (Let's Encrypt/другой)	
1.1.5 Какой IP сервера (публичный)?	

1.2 Базы данных и сервисы

Вопрос	Ваш ответ
1.2.1 Какая БД? (PostgreSQL/MySQL/другая)	
1.2.2 Где БД? (на том же сервере/отдельный)	
1.2.3 Есть ли Redis? (да/нет, где)	
1.2.4 Нужен ли message queue? (RabbitMQ/Kafka/нет)	

1.3 Docker и оркестрация

Вопрос	Ваш ответ
1.3.1 Использовать Docker? (да/нет)	
1.3.2 Использовать Docker Compose? (да/нет)	
1.3.3 Использовать Kubernetes? (да/нет/планируется)	

1.3.4 Есть ли preference по container registry? (Docker Hub/GitHub/self-hosted)

● СЕКЦИЯ 2: NODES ДЛЯ PHASE 1-2 (ОБЯЗАТЕЛЬНО)

Phase 1-2 = без платных функций. Только core + messenger + working nodes.

2.1 Приоритет узлов

Из 20 узлов, какие **5-7 узлов** нужны в Phase 1-2?

Node	Canonical Hostname	Priority (1-5)	Include in Phase 1-2?
projectgeneralsettings	projectgeneralsettings.working.balloo.su		☐
kodegen	kodegen.working.balloo.su		☐
workdocs	workdocs.working.balloo.su		☐
working	working.balloo.su		☐
messenger	messenger.balloo.su		☐
alpha	alpha.balloo.su		☐
admin	admin.balloo.su		☐
Другие	укажите		☐

2.2 Messenger (критично для Phase 1)

Вопрос	Ваш ответ
2.2.1 Real-time или polling? (WebSocket/polling)	
2.2.2 Хранение сообщений? (БД/файлы/память)	
2.2.3 Какие типы сообщений? (text/file/image/voice)	
2.2.4 Максимальный размер файла? (MB)	
2.2.5 Нужно ли шифрование сообщений? (да/нет)	

● СЕКЦИЯ 3: AUTH PROVIDERS ДЛЯ PHASE 1 (ОБЯЗАТЕЛЬНО)

Phase 1 = только 3 провайдера. Какие включаем?

3.1 Выбор провайдеров

Provider	Include in Phase 1?	Notes
----------	---------------------	-------

Provider	Include in Phase 1?	Notes
yandex-id	☐	External OIDC
email-password	☐	Local credentials
phone-3char-code	☐	SMS/bot OTP

3.2 Конфигурация провайдеров

Вопрос	Ваш ответ
3.2.1 Yandex OAuth credentials есть? (client-id/secret)	
3.2.2 SMS provider для OTP? (какой)	
3.2.3 Email server для verification? (SMTP server)	
3.2.4 Пароль политика? (min 8 символов/строгая)	

● СЕКЦИЯ 4: ACCESS ROLES ДЛЯ PHASE 1 (ОБЯЗАТЕЛЬНО)

Какие роли нужны сразу?

4.1 Активация ролей

Role	Include in Phase 1?	Notes
creator-superadmin	☒ Always	Оберюхтин Иван Анатольевич
delegated-node-admin	☐	Per-node delegated
company-staff	☐	NBS-wt employees
alpha-staff	☐	Alpha curators
alpha-volunteer	☐	Alpha testers
sandbox-operator	☐	Working users
public-user	☐	Public access

4.2 Пользователи Phase 1

Вопрос	Ваш ответ
4.2.1 Сколько пользователей в Phase 1? (~10/~50/~100)	
4.2.2 Кто первые пользователи? (сотрудники/тестировщики)	
4.2.3 Нужен ли bulk user import? (да/нет, формат)	

● СЕКЦИЯ 5: CORE PACKAGES (ВАЖНО)

7 core пакетов требуют спецификации.

5.1 Core Packages Usage

Package	Used in Phase 1?	Priority (High/Medium/Low)
core-types	☐	
core-config	☐	
core-i18n	☐	
core-theme	☐	
core-brand	☐	
core-ui	☐	
core-docs-schema	☐	

5.2 Детали core пакетов

Вопрос	Ваш ответ
5.2.1 core-types: сколько примерно типов? (~10/~50/~100)	
5.2.2 core-ui: сколько компонентов? (~10/~30/~50)	
5.2.3 core-i18n: какие языки? (ru/en/другие)	
5.2.4 core-theme: сколько тем? (light/dark/custom)	

🌀 СЕКЦИЯ 6: CODEGEN ПРИОРИТЕТЫ (ВАЖНО)

Что генерировать в первую очередь?

6.1 Codegen Output

Generate What?	Priority (1-5)	Notes
TypeScript types		Из core-types contracts
API routes		Из endpoint specs
React components		Из design system
Configuration files		Из schemas
Documentation pages		Из contracts
Test files		Из contracts

6.2 Codegen Workflow

Вопрос	Ваш ответ
6.2.1 Запуск codegen? (manual/automatic/CI)	
6.2.2 Commit'ить сгенерированный код? (да/нет)	
6.2.3 Где хранить templates? (templates/ папка)	

🌀 СЕКЦИЯ 7: TESTING STRATEGY (ВАЖНО)

Минимум тестов для Phase 1.

7.1 Test Types

Test Type	Include in Phase 1?	Framework Preference
Unit tests	☐	Jest/Vitest
Integration tests	☐	Supertest/Testing Library
E2E tests	☐	Playwright/Cypress
API tests	☐	Jest/Supertest

7.2 Coverage Target

Вопрос	Ваш ответ
7.2.1 Minimum coverage для Phase 1? (20%/30%/50%)	
7.2.2 Критичные модули для тестов? (messenger/auth/core)	

🌀 СЕКЦИЯ 8: DEPLOYMENT WORKFLOW (ЖЕЛАТЕЛЬНО)

Как разворачивать на Ubuntu.

8.1 Deployment Steps

Вопрос	Ваш ответ
8.1.1 Manual или automated deploy?	
8.1.2 Есть ли deployment scripts? (bash/Ansible/другое)	
8.1.3 Время на deploy? (минуты)	
8.1.4 Rollback план? (да/нет)	

8.2 Monitoring

Вопрос	Ваш ответ
--------	-----------

Вопрос	Ваш ответ
8.2.1 Monitor uptime? (да/нет)	
8.2.2 Error tracking? (Sentry/self-hosted/нет)	
8.2.3 Log aggregation? (ELK/papertrail/файлы)	

○ СЕКЦИЯ 9: LEGACY CLEANUP (НЕОБЯЗАТЕЛЬНО)

Можно отложить на Phase 3.

9.1 Legacy Directories

Directory	Delete in Phase 1?	Migrate first?
docs-contracts/	☐	☐
docs-migration/	☐	☐
docs-site/	☐	☐
workdocs/legacy-*	☐	☐

СЕКЦИЯ 10: ДОПОЛНИТЕЛЬНЫЕ ТРЕБОВАНИЯ

10.1 Специфичные требования

[Опишите любые дополнительные требования, которые не вошли в опросник]

10.2 Ограничения

[Опишите ограничения: бюджет, время, ресурсы, compliance]

10.3 Риски

[Опишите известные риски: технические, организационные]

☒ КАК ЗАПОЛНЯТЬ

Вариант 1: Быстрый (15 минут)

Заполните только  **Красные секции** (1-4):

- Серверная инфраструктура
- Nodes для Phase 1-2
- Auth providers
- Access roles

Результат: Могу создать deployment инструкцию и базовую систему.

Вариант 2: Полный (30 минут)

Заполните  **Красные** +  **Оранжевые секции** (1-7):

- Всё из быстрого + core packages + codegen + testing

Результат: Могу создать полную систему с тестами и codegen.

Вариант 3: Максимальный (45 минут)

Заполните **все секции** (1-10):

- Всё из полного + deployment workflow + legacy cleanup + дополнительные требования

Результат: Максимально полная спецификация для production-ready системы.

СЛЕДУЮЩИЕ ШАГИ

После заполнения:

1. **Вы отправляете ответы** (можно по частям, начиная с  секций)
2. **Я создаю единый спес файл** (**BALLOO_BUILD_SPEC.md**)
3. **Вы даёте команду** «собери и разработай проект»
4. **Я выполняю:**
 - 4.1: Документация completion
 - 4.2: Unified spec файл
 - 4.3: Codegen и implementation (Phase 1-2)
 - 4.4: Ubuntu deployment инструкция

ПРИЛОЖЕНИЯ

A: Список узлов (20 total)

```
Group A (Privileged, 4):
- projectgeneralsettings.working.balloo.su
- codegen.working.balloo.su
- pilot-future.working.balloo.su
- nodes-switcher.working.balloo.su
```

```
Group B (Company, 2):
- workdocs.working.balloo.su
```


- admin.balloo.su

Group C (Alpha, 3):

- alpha.balloo.su
- apps.alpha.balloo.su
- 2commands.alpha.balloo.su

Group D (Sandbox, 9):

- working.balloo.su
- api.working.balloo.su
- files.working.balloo.su
- docs.working.balloo.su
- future.working.balloo.su
- admin.working.balloo.su
- workers.working.balloo.su
- about.working.balloo.su
- apps.working.balloo.su

Group E (Production, 2):

- balloo.su
- messenger.balloo.su

B: Список провайдеров (6 total)

Phase 1 (3):

- yandex-id (External OIDC)
- email-password (Local credentials)
- phone-3char-code (Phone OTP)

Phase 2 (3):

- gosuslugi (State identity)
- max (Messenger code via MAX bot)
- qr-code (Device transfer)

C: Список ролей (7 total)

- creator-superadmin (L10) – Always active
- delegated-node-admin (L8) – Per-node delegation
- company-staff (L6) – NBS-wt employees
- alpha-staff (L5) – Alpha curators
- alpha-volunteer (L4) – Alpha testers
- sandbox-operator (L3) – Working users
- public-user (L1) – Public access

🔗 **Balloo - Проверни общение!**

Создано: 2026-06-14

Версия: 2.0.0 (Build Spec Edition)

Статус: Active

Автор: Koda (NLP-Core-Team)

КАК ИСПОЛЬЗОВАТЬ

Этот опросник разделён на секции. Вы можете отвечать на вопросы по мере готовности.

Формат ответов:

Секция: [Название]
Вопрос: [Номер]
Ответ: [Ваш ответ]
Дополнительно: [Файлы, ссылки, примеры]

Приоритеты:

-  Critical — обязательно для completion
 -  High — важно для production
 -  Medium — желательно для improvement
 -  Low — nice to have
-

СЕКЦИЯ 1: MODULE DOCUMENTATION (CRITICAL)

1.1 Core Packages Documentation

Контекст: 7 core пакетов имеют только pattern, нужны полные summary + contract.

Вопрос 1.1.1: core-types

Приоритет:  Critical

Вопрос: Какие основные типы экспортирует core-types?

Подвопросы:

- Сколько примерно типов (10, 50, 100+)?
- Какие категории типов (common, node, messenger, admin, other)?
- Есть ли типы для всех приложений или только для некоторых?
- Какие типы используются чаще всего?

Формат ответа:

```typescript

// Пример

export type ID = string;

export interface NodeConfig { ... }

```
export interface Message { ... }
// ... ещё ~50 типов
```

#### Вопрос 1.1.2: core-config

Приоритет: ● Critical

Вопрос: Как работает core-config?

Подвопросы:

- Какие конфигурации управляет (app, service, node)?
- Как загружаются конфиги (env, file, runtime)?
- Есть ли валидация конфигов?
- Какие обязательные поля в конфигах?

Формат ответа:

```
{
 configTypes: ["app", "service", "node"],
 loadMethod: "env+file",
 validation: true/false,
 requiredFields: ["nodeId", "nodeType", ...]
}
```

#### Вопрос 1.1.3: core-i18n

Приоритет: ● Critical

Вопрос: Как работает интернационализация?

Подвопросы:

- Какие языки поддерживаются (ru, en, other)?
- Где хранятся переводы (files, DB, external)?
- Есть ли fallback язык?
- Как переключается язык (runtime, build-time)?

Формат ответа:

```
{
 languages: ["ru", "en"],
 storage: "files",
 fallback: "ru",
}
```

```
switchMethod: "runtime"
}
```

#### Вопрос 1.1.4: core-theme

Приоритет: ☒ High

Вопрос: Как работает система тем?

Подвопросы:

- Сколько тем (light, dark, custom)?
- Какие переменные тем (colors, spacing, typography)?
- Как применяется тема (CSS variables, context, other)?
- Есть ли кастомизация пользователем?

Формат ответа:

```
{
themes: ["light", "dark"],
variables: ["colors", "spacing", "typography"],
application: "CSS variables",
userCustomization: true/false
}
```

#### Вопрос 1.1.5: core-brand

Приоритет: ☒ High

Вопрос: Что включает brand identity?

Подвопросы:

- Логотипы (сколько версий, форматы)?
- Цвета бренда (primary, secondary, accent)?
- Типографика (шрифты, размеры)?
- Brand guidelines документированы?

Формат ответа:

```
{
logos: ["full", "icon", "text"],
colors: { primary: "#...", secondary: "#..." },
fonts: ["font1", "font2"],
guidelines: true/false
}
```

#### #### Вопрос 1.1.6: core-ui

Приоритет:  Critical

Вопрос: Какие UI компоненты предоставляет?

Подвопросы:

- Сколько компонентов (~10, ~50, ~100)?
- Какие категории (buttons, forms, layout, data-display)?
- Есть ли документация компонентов (Storybook)?
- Какие компоненты используются во всех приложениях?

Формат ответа:

```
{
totalComponents: ~30,
categories: ["buttons", "forms", "layout", "data-display"],
documentation: "Storybook/none",
sharedComponents: ["Button", "Input", "Card", ...]
}
```

#### #### Вопрос 1.1.7: core-docs-schema

Приоритет:  Medium

Вопрос: Что определяет docs-schema?

Подвопросы:

- Schema для каких документов (Markdown, JSON, other)?
- Какие обязательные поля в frontmatter?
- Есть ли валидация документов?
- Используется ли для docgen?

Формат ответа:

```
{
documentTypes: ["Markdown", "JSON"],
requiredFrontmatter: ["title", "description", "version"],
validation: true/false,
usedForDocgen: true/false
}
```

---

### ### 1.2 Application Documentation

**\*\*Контекст:\*\*** 4 приложения (messenger, admin-portal, desktop, mobile) требуют summary + contract.

#### #### Вопрос 1.2.1: messenger

Приоритет:  Critical

Вопрос: Как работает messenger?

Подвопросы:

- Какие основные функции (chat, file-share, voice, other)?
- Real-time или polling?
- Хранение сообщений (local, server, both)?
- Какие API endpoints уже реализованы?
- Сколько пользователей (ожидаемое количество)?

Формат ответа:

```
{
features: ["chat", "file-share", ...],
realtime: true/false,
storage: "server+local",
endpoints: ["GET /messages", "POST /messages", ...],
expectedUsers: ~100
}
```

#### #### Вопрос 1.2.2: admin-portal

Приоритет:  Critical

Вопрос: Что можно делать через admin-portal?

Подвопросы:

- Управление чем (users, nodes, services, other)?
- Какие роли/permissions (admin, operator, viewer)?
- Есть ли audit logging?
- Какие метрики отображаются?

Формат ответа:

```
{
```

```
management: ["users", "nodes", "services"],
roles: ["admin", "operator", "viewer"],
auditLogging: true/false,
metrics: ["uptime", "errors", "performance"]
}
```

#### Вопрос 1.2.3: desktop

Приоритет:  High

Вопрос: Что делает desktop приложение?

Подвопросы:

- Electron или другой фреймворк?
- Какие функции (local control, monitoring, other)?
- Работает offline?
- Интеграция с OS (notifications, tray, other)?

Формат ответа:

```
{
framework: "Electron",
functions: ["control", "monitoring"],
offline: true/false,
osIntegration: ["notifications", "tray"]
}
```

#### Вопрос 1.2.4: mobile

Приоритет:  High

Вопрос: Что делает mobile приложение?

Подвопросы:

- React Native или другой фреймворк?
- iOS, Android, или both?
- Какие функции (messenger, monitoring, other)?
- Push notifications?

Формат ответа:

```
{
framework: "React Native",
platforms: ["iOS", "Android"],
}
```

```
functions: ["messenger", "notifications"],
pushNotifications: true/false
}
```

---

### ### 1.3 Infrastructure Documentation

**\*\*Контекст:\*\*** android-service, node-system, summary-docs требуют completion.

#### #### Вопрос 1.3.1: android-service

Приоритет:  Critical

Вопрос: Что делает android-service?

Подвопросы:

- Какие API предоставляет?
- Для каких Android клиентов?
- Authentication как работает?
- Данные синхронизируются?

Формат ответа:

```
{
apis: ["sync", "config", "status"],
clients: ["android-app"],
authentication: "token-based",
sync: true/false
}
```

#### #### Вопрос 1.3.2: node-system

Приоритет:  High

Вопрос: Какие node contracts самые важные?

Подвопросы:

- Какие контракты используются чаще (NodeTree, NodeRoles, other)?
- Есть ли конфликты между контрактами?
- Все ли узлы соответствуют контрактам?
- Требуется ли валидация nodes?



Формат ответа:

```
{
criticalContracts: ["NodeTreeContract", "NodeRolesContract"],
conflicts: true/false,
allNodesCompliant: true/false,
validationNeeded: true/false
}
```

#### Вопрос 1.3.3: summary-docs

Приоритет: ☒ Medium

Вопрос: Как используется summary-docs?

Подвопросы:

- Кто основная аудитория (AI, developers, both)?
- Как часто обновляется?
- Есть ли automated doc generation?
- Web reader достаточно или нужно больше?

Формат ответа:

```
{
audience: ["AI", "developers"],
updateFrequency: "manual/automated",
autoGeneration: true/false,
webReaderSufficient: true/false
}
```

---

## СЕКЦИЯ 2: CODEGEN (☒ HIGH)

### 2.1 Codegen Requirements

#### Вопрос 2.1.1: Codegen Priority

Приоритет: ☒ Critical

Вопрос: Что генерировать в первую очередь?

Варианты:

- ☐ Type definitions из MODULE\_CONTRACT

- ☐ API routes из endpoint specs
- ☐ Component scaffolding из templates
- ☐ Configuration files из schemas
- ☐ Documentation из contracts
- ☐ Deployment configs из node-presence

Формат ответа:

priority: ["types", "api-routes", ...]

#### Вопрос 2.1.2: Codegen Input

Приоритет: ☒ High

Вопрос: Какие источники для codegen?

Подвопросы:

- MODULE\_CONTRACT\_\*.md достаточно?
- Нужны ли дополнительные specs?
- JSON schema для конфигов?
- OpenAPI specs для API?

Формат ответа:

```
{
 primarySources: ["MODULE_CONTRACT_*.md"],
 additionalSpecs: ["JSON schema", "OpenAPI"],
 openApiForApis: true/false
}
```

#### Вопрос 2.1.3: Codegen Output

Приоритет: ☒ High

Вопрос: Что генерировать?

Подвопросы:

- TypeScript types/interfaces?
- API route handlers?
- React components?
- Configuration files?
- Documentation pages?
- Test files?

Формат ответа:

```
{
generateTypes: true/false,
generateApiRoutes: true/false,
generateComponents: true/false,
generateConfigs: true/false,
generateDocs: true/false,
generateTests: true/false
}
```

#### Вопрос 2.1.4: Codegen Templates

Приоритет: ☒ Medium

Вопрос: Есть ли предпочтения по templates?

Подвопросы:

- Handlebars, EJS, или string templates?
- Один template на тип модуля?
- Custom templates для каждого модуля?
- Где хранить templates?

Формат ответа:

```
{
templateEngine: "Handlebars/EJS/string",
oneTemplatePerType: true/false,
customTemplates: true/false,
templateLocation: "templates/"
}
```

---

## СЕКЦИЯ 3: TESTING (☒ CRITICAL)

### 3.1 Test Strategy

#### Вопрос 3.1.1: Test Priority

Приоритет: ☒ Critical

Вопрос: Что тестировать в первую очередь?

Варианты:

- ☐ Core packages (core-types, core-config, ...)
- ☐ API endpoints (messenger, admin-portal)
- ☐ UI components (core-ui, app components)
- ☐ Integration between modules
- ☐ E2E flows (user scenarios)

Формат ответа:

priority: ["core-packages", "api-endpoints", ...]

#### Вопрос 3.1.2: Test Framework

Приоритет: ☒ High

Вопрос: Какие фреймворки предпочитаете?

Подвопросы:

- Unit tests: Jest, Vitest, other?
- Integration: Supertest, Testing Library?
- E2E: Playwright, Cypress, other?
- Coverage: Istanbul, c8?

Формат ответа:

```
{
unit: "Jest/Vitest/other",
integration: "Supertest/Testing Library",
e2e: "Playwright/Cypress",
coverage: "Istanbul/c8"
}
```

#### Вопрос 3.1.3: Test Coverage Target

Приоритет: ☐ Medium

Вопрос: Какой target по coverage?

Подвопросы:

- Short-term target (3 месяца)?
- Medium-term target (6 месяцев)?
- Long-term target (12 месяцев)?
- Minimum acceptable coverage?

Формат ответа:

```
{
shortTerm: "30%",
mediumTerm: "50%",
longTerm: "80%",
minimumAcceptable: "20%"
}
```

#### Вопрос 3.1.4: Existing Tests

Приоритет: ☒ High

Вопрос: Есть ли уже тесты?

Подвопросы:

- Где расположены тесты?
- Сколько тестов примерно?
- Какие типы тестов есть?
- Почему мало тестов (время, priority, other)?

Формат ответа:

```
{
testLocations: ["packages//tests", "apps//tests"],
approximateCount: ~50,
testTypes: ["unit"],
reasonForFew: "priority/time"
}
```

---

## СЕКЦИЯ 4: CI/CD ( ☒ CRITICAL )

### 4.1 CI/CD Requirements

#### Вопрос 4.1.1: CI Pipeline

Приоритет: ☒ Critical

Вопрос: Что должно быть в CI?

Варианты:

- ☐ Lint check (ESLint)

- ☐ Type check (TypeScript)
- ☐ Test run (Jest)
- ☐ Build verification
- ☐ Security scan
- ☐ Documentation generation

Формат ответа:

cdSteps: ["lint", "typecheck", "test", "build", ...]

#### Вопрос 4.1.2: CD Pipeline

Приоритет: ☒ High

Вопрос: Что должно быть в CD?

Варианты:

- ☐ Deploy to staging on PR merge
- ☐ Deploy to production on tag
- ☐ Manual approval for production
- ☐ Rollback on failure
- ☐ Notification on deploy

Формат ответа:

cdSteps: ["staging-on-merge", "production-on-tag", ...]

#### Вопрос 4.1.3: Environments

Приоритет: ☒ High

Вопрос: Какие environments нужны?

Подвопросы:

- Development (local)?
- Staging (pre-production)?
- Production?
- Per-PR preview?

Формат ответа:

```
{
 environments: ["development", "staging", "production"],
 perPrPreview: true/false
}
```

#### #### Вопрос 4.1.4: Deployment Targets

Приоритет:  High

Вопрос: Куда deploy'ить?

Подвопросы:

- VPS/VM (какой provider)?
- Container (Docker, Kubernetes)?
- Serverless (Vercel, Netlify)?
- On-premise (свои серверы)?

Формат ответа:

```
{
primaryTarget: "VPS/Docker/Serverless/On-prem",
provider: "DigitalOcean/AWS/self-hosted",
containerized: true/false,
orchestration: "none/Docker Compose/K8s"
}
```

---

## ## СЕКЦИЯ 5: DEPLOYMENT ( HIGH)

### ### 5.1 Deployment Strategy

#### #### Вопрос 5.1.1: Current Deployment

Приоритет:  High

Вопрос: Как deploy'ите сейчас?

Подвопросы:

- Manual или automated?
- Есть ли deployment scripts?
- Сколько времени занимает deploy?
- Частые ли проблемы при deploy?

Формат ответа:

```
{
currentMethod: "manual/semi-auto/auto",
scriptsExist: true/false,
deployTime: "~30 min",
}
```

```
frequentIssues: true/false
}
```

#### #### Вопрос 5.1.2: Docker Usage

Приоритет: ☒ Medium

Вопрос: Используется ли Docker?

Подвопросы:

- Dockerfile для каждого приложения?
- Docker Compose для оркестрации?
- Docker registry (какой)?
- Container tagging strategy?

Формат ответа:

```
{
dockerfiles: "all/some/none",
dockerCompose: true/false,
registry: "Docker Hub/self-hosted",
taggingStrategy: "semver/latest"
}
```

#### #### Вопрос 5.1.3: Service Dependencies

Приоритет: ☒ High

Вопрос: Какие service dependencies?

Подвопросы:

- Database (какая, где)?
- Cache (Redis, other)?
- Message queue (RabbitMQ, Kafka)?
- External APIs (какие)?

Формат ответа:

```
{
database: "PostgreSQL on work_server",
cache: "Redis on work_server",
messageQueue: "none",
externalApis: ["GitHub", "Tailscale"]
}
```



---

## ## СЕКЦИЯ 6: LEGACY CLEANUP ( ☒ MEDIUM)

### ### 6.1 Legacy Migration

#### #### Вопрос 6.1.1: Legacy Directories

Приоритет: ☒ Medium

Вопрос: Что делать с legacy директориями?

Директории:

- docs-contracts/
- docs-migration/
- docs-site/
- workdocs/
- docs/

Подвопросы:

- Есть ли уникальные документы в legacy?
- Всё ли уже мигрировано в SUMMARY\_DOCS?
- Можно ли удалить legacy?
- Есть ли ссылки на legacy?

Формат ответа:

```
{
uniqueDocsInLegacy: true/false,
migrationComplete: true/false,
canDelete: true/false,
linksExist: true/false
}
```

#### #### Вопрос 6.1.2: Redirect Strategy

Приоритет: ☐ Low

Вопрос: Как обрабатывать старые ссылки?

Подвопросы:

- Нужны ли redirects со старых путей?
- Где хранить redirect map?

- Автоматизировать redirects?

Формат ответа:

```
{
redirectsNeeded: true/false,
redirectMapLocation: "SUMMARY_DOCS/ROUTING.json",
automateRedirects: true/false
}
```

---

## СЕКЦИЯ 7: INTEGRATION ( ☒ MEDIUM)

### 7.1 Module Integration

#### Вопрос 7.1.1: Module Dependencies

Приоритет: ☒ Medium

Вопрос: Какие зависимости между модулями критичны?

Подвопросы:

- Какие модули зависят от core-types?
- Какие модули зависят от core-config?
- Есть ли circular dependencies?
- Какие зависимости самые проблемные?

Формат ответа:

```
{
criticalDependencies: [
{ from: "messenger", to: "core-types" },
{ from: "admin-portal", to: "core-ui" }
],
circularDependencies: true/false,
problematicDependencies: [...]
}
```

#### Вопрос 7.1.2: API Versioning

Приоритет: ☐ Low

Вопрос: Как версионировать API?

Подвопросы:

- Текущая стратегия versioning?
- semver для API?
- Deprecation policy?
- Migration path для breaking changes?

Формат ответа:

```
{
currentStrategy: "none/semver/other",
useSemver: true/false,
deprecationPolicy: "documented/none",
migrationPath: true/false
}
```

---

## СЕКЦИЯ 8: AI/CODEGEN INTEGRATION (🔴 HIGH)

### 8.1 AI Workflow

#### Вопрос 8.1.1: AI Usage Pattern

Приоритет: 🔴 High

Вопрос: Как AI должен использовать документацию?

Подвопросы:

- AI читает contracts перед codegen?
- AI генерирует code из contracts?
- AI обновляет docs из code?
- AI валидирует contracts?

Формат ответа:

```
{
aiReadsContracts: true/false,
aiGeneratesCode: true/false,
aiUpdatesDocs: true/false,
aiValidatesContracts: true/false
}
```

#### Вопрос 8.1.2: Codegen Workflow

Приоритет: ☒ High

Вопрос: Как должен работать codegen?

Подвопросы:

- Запуск по команде или автоматически?
- Какие файлы генерировать?
- Где хранить сгенерированные файлы?
- Commit'ить сгенерированный код?

Формат ответа:

```
{
trigger: "manual/automatic",
generatedFiles: ["types", "routes", "components"],
outputLocation: "generated/",
commitGenerated: true/false
}
```

```
Вопрос 8.1.3: Contract Validation
```

Приоритет: ☐ Medium

Вопрос: Валидировать ли contracts?

Подвопросы:

- Валидация syntax contracts?
- Валидация completeness?
- Валидация consistency между contracts?
- Автоматическая валидация в CI?

Формат ответа:

```
{
validateSyntax: true/false,
validateCompleteness: true/false,
validateConsistency: true/false,
validateInCI: true/false
}
```

```

```

```
СЕКЦИЯ 9: METRICS & MONITORING (☐ LOW)
```

```
9.1 Success Metrics
```

#### #### Вопрос 9.1.1: Project Metrics

Приоритет: ○ Low

Вопрос: Какие метрики важны?

Варианты:

- ☐ Test coverage %
- ☐ Documentation coverage %
- ☐ Build time
- ☐ Deploy frequency
- ☐ Error rate
- ☐ User satisfaction

Формат ответа:

importantMetrics: ["coverage", "build-time", ...]

#### #### Вопрос 9.1.2: Monitoring

Приоритет: ○ Low

Вопрос: Что мониторить?

Подвопросы:

- Uptime сервисов?
- Performance метрики?
- Error tracking?
- User analytics?

Формат ответа:

```
{
monitorUptime: true/false,
monitorPerformance: true/false,
errorTracking: "Sentry/self-hosted/none",
userAnalytics: true/false
}
```

---

##  NOTES SECTION

### Дополнительные комментарии:

[Место для ваших заметок, вопросов, комментариев]

### Файлы для прикрепления:

[Перечислите файлы, которые могут помочь: примеры кода, схемы, диаграммы]

### Контакты для уточнений:

[Кто может ответить на дополнительные вопросы]

---

## ☒ NEXT STEPS

После заполнения:

1. **\*\*Приоритизация\*\*** – определим priority задач
2. **\*\*Планирование\*\*** – создам roadmap на основе ответов
3. **\*\*Выполнение\*\*** – реализую critical tasks
4. **\*\*Валидация\*\*** – проверю результаты

---

**\*\*🔗 Balloo - Переверни общение!\*\***

**\*\*Создано:\*\*** 2026-06-14

**\*\*Версия:\*\*** 1.0.0

**\*\*Статус:\*\*** Active

**\*\*Автор:\*\*** Koda (NLP-Core-Team)